

TD-MPC2 Baseline 复现报告

环境配置、踩坑记录、验证结果与核心源码走读

xuyidong

2026-07-05

Abstract

本文记录在 A100 服务器上复现 TD-MPC2 baseline 的过程。复现目标是先跑通官方 TD-MPC2 的 dog-run baseline, 包括仓库克隆、依赖环境、MuJoCo/DMControl 验证、官方 checkpoint 评估、短步数训练 smoke test, 以及后续正式复现实验命令。文档只覆盖原始 baseline 代码路径, 不讨论后续新增的 Planner Distillation 变体。

1 复现目标与代码位置

本次复现的 baseline 是 TD-MPC2 原始实现, 任务为 DMControl 的 dog-run。代码根目录为:

```
1 /home/xuyidong/tdmpc2/tdmpc2
```

baseline 只使用原始文件:

- train.py: 训练入口。
- evaluate.py: 评估入口。
- tdmpc2.py: TD-MPC2 agent, 包括 planning 和 update。
- common/world_model.py: encoder、dynamics、reward、policy prior、Q ensemble。
- common/buffer.py: TorchRL replay buffer。
- trainer/online_trainer.py: 单任务在线 RL 训练循环。

2 硬件与软件环境

服务器检测结果如下:

项目	结果
GPU	8 张 NVIDIA A100-SXM4-80GB
单卡显存	81920 MiB
NVIDIA Driver	580.159.03
系统 CUDA 显示	13.0
Conda 环境	tdmpc2
Python	3.9.23
PyTorch	2.6.0.dev20241112
MuJoCo Python	3.1.2
CUDA 可用	True
可见 GPU 数量	8

激活环境命令:

```
1 source ~/miniforge3/etc/profile.d/conda.sh
2 conda activate tdmpc2
3 cd ~/tdmpc2/tdmpc2
4 export MUJOCO_GL=egl
```

其中 MUJOCO_GL=egl 对无显示器服务器很重要，否则 MuJoCo/DMControl 渲染和环境初始化可能出问题。

3 环境配置过程与遇到的坑

3.1 仓库克隆

最初在终端执行：

```
1 git clone https://github.com/nicklashansen/tdmpc2
```

曾停在 Cloning into 'tdmpc2'.... 判断为服务器到 GitHub 连接握手阶段不稳定。重新发起 clone 后成功，当前仓库位于：

```
1 /home/xuyidong/tdmpc2
```

3.2 Docker 权限不可用

仓库推荐 Docker，但当前用户不在 docker 组，直接运行 docker ps 报：

```
1 permission denied while trying to connect to the docker API at unix:///var/run/docker.
   sock
```

sudo docker 也需要密码。因此最后采用用户目录下的 Miniforge + mamba 方案，不依赖管理员权限。

3.3 Conda 环境创建中的 nightly 包问题

仓库的 docker/environment.yaml 里包含：

```
1 tensordict-nightly==2025.1.1
2 torchrl-nightly==2025.1.1
```

其中 torchrl-nightly==2025.1.1 当前 pip 索引找不到，报错：

```
1 ERROR: Could not find a version that satisfies the requirement torchrl-nightly==2025.1.1
2 ERROR: No matching distribution found for torchrl-nightly==2025.1.1
```

解决办法是保留 conda 已创建出的 tdmpc2 环境，然后补装正式版：

```
1 python -m pip install torchrl==0.6.0 tensordict==0.6.2
```

最终核心包可正常导入：

```
1 python -c "import torch, mujoco, dm_control, tensordict, torchrl, kornia; print(torch.
   __version__); print(torch.cuda.is_available())"
```

输出显示 CUDA 可用，且可见 8 张 GPU。

3.4 Hydra launcher 缺失

原始 config.yaml 默认包含：

```
1 defaults:
2   - override hydra/launcher: submitit_local
```

但环境里没有 `hydra-submitit-launcher` 时，直接运行会报：

```
1 In 'hydra/config': Could not find 'hydra/launcher/submitit_local'
2 Available options in 'hydra/launcher': basic
```

本次 baseline 复现统一在命令行显式覆盖：

```
1 hydra/launcher=basic
```

这样不需要修改原始 `config.yaml`。

3.5 Hugging Face checkpoint 下载超时

下载官方 `dog-run-1.pt` 时，`wget` 曾连接 Hugging Face 超时，并留下 0 字节文件：

```
1 Connecting to huggingface.co ... failed: Connection timed out.
```

解决方式是删除空文件后，用带重试的 `curl -L` 下载；最终 checkpoint 大小约 32 MB：

```
1 mkdir -p ~/tdmpc2/checkpoints
2 curl -L --retry 5 --retry-delay 3 \
3   -o ~/tdmpc2/checkpoints/dog-run-1.pt \
4   https://huggingface.co/nicklashansen/tdmpc2/resolve/main/dmcontrol/dog-run-1.pt
5 ls -lh ~/tdmpc2/checkpoints/dog-run-1.pt
```

3.6 日志不刷新

使用 `nohup python train.py > log 2>&1 &` 时，日志长时间只有：

```
1 nohup: ignoring input
```

原因是 Python 输出被缓冲。后续后台训练建议使用 `python -u`：

```
1 nohup python -u train.py ... > train.log 2>&1 &
```

3.7 为什么 eval.csv 一直是 step 0

`eval.csv` 不是训练进度条，只在评估时写入。如果设置：

```
1 steps=10000 eval_freq=10000
```

则只会写两次：`step=0` 和 `step=10000`。中间训练步不会写入该 CSV。

4 Baseline 配置

原始配置中的关键默认值如下：

```
1 task: dog-run
2 obs: state
3 eval_episodes: 10
4 eval_freq: 50000
5 steps: 10_000_000
6 batch_size: 256
7 buffer_size: 1_000_000
8 mpc: true
9 iterations: 6
10 num_samples: 512
11 num_elites: 64
12 num_pi_trajs: 24
13 horizon: 3
14 model_size: ???
```

```

15 compile: true
16 save_video: true
17 seed: 1

```

Listing 1: config.yaml 中的 baseline 关键配置

README 中给 dog-run 的训练示例是：

```

1 python train.py task=dog-run steps=7000000

```

因此正式复现建议至少使用 700 万环境步；本文中的 1 万步实验仅用于 smoke test。

5 官方 Checkpoint 评估验证

下载官方 checkpoint 后，使用原始 evaluate.py 验证：

```

1 cd ~/tdmpc2/tdmpc2
2 export MUJOOCO_GL=egl
3
4 python evaluate.py \
5     hydra/launcher=basic \
6     task=dog-run \
7     model_size=5 \
8     checkpoint=~/.tdmpc2/checkpoints/dog-run-1.pt \
9     eval_episodes=1 \
10    save_video=true \
11    compile=false

```

得到结果：

```

1 Task: dog-run
2 Model size: 5
3 Checkpoint: /home/xuyidong/tdmpc2/checkpoints/dog-run-1.pt
4 Episode length: 500
5 Discount factor: 0.99
6 Evaluating agent on dog-run:
7   dog-run           R: 762.2   S: 0.00

```

视频输出：

```

1 ~/.tdmpc2/tdmpc2/logs/dog-run/1/default/videos/dog-run-0.mp4

```

这说明 baseline 的环境、模型加载、planning、MuJoCo 渲染链路均已跑通。

6 Baseline 训练 Smoke Test

为了验证训练流程，运行了 1 万步 baseline：

```

1 CUDA_VISIBLE_DEVICES=0 nohup python -u train.py \
2     hydra/launcher=basic \
3     task=dog-run \
4     model_size=5 \
5     steps=10000 \
6     seed=1 \
7     exp_name=baseline_10k_seed1 \
8     eval_freq=10000 \
9     save_video=false \
10    enable_wandb=false \
11    compile=false \
12    > ~/.tdmpc2/train_baseline_10k.log 2>&1 &

```

评估 CSV 为:

```
1 step,episode_reward
2 0.0,7.995296478271484
3 10000.0,6.3938422203063965
```

Table 2: Baseline 1 万步 smoke test 结果

Step	Episode Reward
0	7.9953
10000	6.3938

该结果很低, 属于正常现象: 1 万环境步对 TD-MPC2 的 dog-run 太短, 只能验证训练流程可运行, 不能代表算法最终性能。

7 Baseline 14 万步训练曲线

除 1 万步 smoke test 外, 还启动了一个计划 30 万步的 baseline 训练:

```
1 CUDA_VISIBLE_DEVICES=0 nohup python -u train.py \
2   hydra/launcher=basic \
3   task=dog-run \
4   model_size=5 \
5   steps=300000 \
6   seed=1 \
7   exp_name=baseline_300k_seed1 \
8   eval_freq=1000 \
9   eval_episodes=5 \
10  save_video=false \
11  enable_wandb=false \
12  compile=false \
13  > ~/tdmpc2/train_baseline_300k.log 2>&1 &
```

该实验的 eval.csv 当前记录到 154000 步, 共 155 个评估点。每个评估点使用 eval_episodes=5, 评估频率为每 1000 环境步一次。关键结果如下:

Table 3: Baseline 14 万余步训练曲线摘要

指标	数值
最后记录步数	154000
最后记录 reward	134.0047
当前最高 reward	157.2973
最高 reward 对应步数	151000
评估点数量	155

从曲线看, baseline 在约 100k 步后开始明显高于 1 万步 smoke test 的水平, 154k 步时 reward 为 134.0, 当前最高点为 151k 步的 157.3。这个结果已经比 1 万步更有参考意义, 但仍不能替代 README 推荐的 700 万步正式复现。

8 正式 Baseline 复现命令

正式复现实验建议使用 README 中的 dog-run 训练步数, 即 700 万步:

```
1 source ~/miniforge3/etc/profile.d/conda.sh
2 conda activate tdmpc2
```

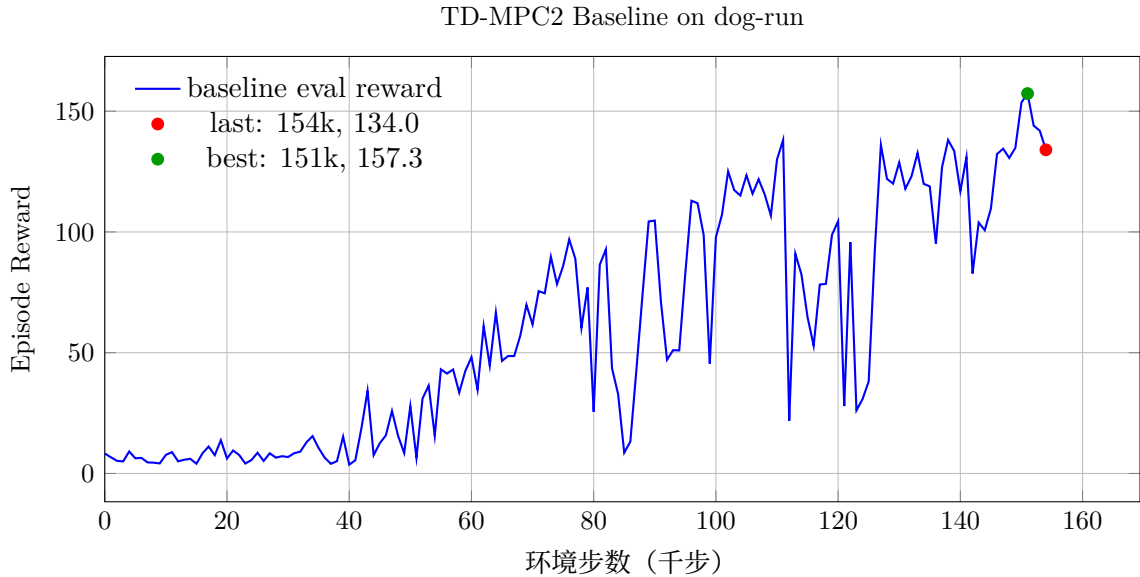


Figure 1: Baseline 从 0 到 154k 环境步的评估 reward 曲线。该曲线说明训练已经超过 14 万步，并且 reward 从初始约 8 提升到百级水平；但距离官方 checkpoint 的 762.2 仍有明显差距，说明仍属于较早训练阶段。

```

3 cd ~/tdmpc2/tdmpc2
4 export MUJOCO_GL=egl
5
6 CUDA_VISIBLE_DEVICES=0 nohup python -u train.py \
7   hydra/launcher=basic \
8   task=dog-run \
9   model_size=5 \
10  steps=7000000 \
11  seed=1 \
12  exp_name=baseline_7m_seed1 \
13  eval_freq=50000 \
14  eval_episodes=10 \
15  save_video=false \
16  enable_wandb=false \
17  compile=false \
18  > ~/tdmpc2/train_baseline_7m_seed1.log 2>&1 &

```

输出位置：

```
1 ~/tdmpc2/tdmpc2/logs/dog-run/1/baseline_7m_seed1/
```

评估曲线：

```
1 ~/tdmpc2/tdmpc2/logs/dog-run/1/baseline_7m_seed1/eval.csv
```

最终模型：

```
1 ~/tdmpc2/tdmpc2/logs/dog-run/1/baseline_7m_seed1/models/final.pt
```

9 Baseline 核心源码走读

9.1 训练入口：train.py

训练入口负责构建环境、agent、buffer 和 logger。关键逻辑位于 train.py 第 52–60 行：

```
1 trainer_cls = OfflineTrainer if cfg.multitask else OnlineTrainer
```

```

2 trainer = trainer_cls(
3     cfg=cfg,
4     env=make_env(cfg),
5     agent=TDMP2(cfg),
6     buffer=Buffer(cfg),
7     logger=Logger(cfg),
8 )
9 trainer.train()

```

Listing 2: train.py: 构建 trainer 并启动训练

单任务 dog-run 会使用 OnlineTrainer。

9.2 Agent 初始化: tdm2.py

原始 TD-MPC2 agent 固定使用当前进程可见的 cuda:0。这也是为什么一个进程不会自动用 8 张 GPU；如果设置 CUDA_VISIBLE_DEVICES=3，代码里的 cuda:0 会映射到物理 GPU 3。

```

1 self.device = torch.device('cuda:0')
2 self.model = WorldModel(cfg).to(self.device)
3 self.optim = torch.optim.Adam([
4     {'params': self.model._encoder.parameters(), 'lr': self.cfg.lr*self.cfg.enc_lr_scale},
5     {'params': self.model._dynamics.parameters()},
6     {'params': self.model._reward.parameters()},
7     {'params': self.model._termination.parameters() if self.cfg.episodic else []},
8     {'params': self.model._qs.parameters()},
9     {'params': self.model._task_emb.parameters() if self.cfg.multitask else []}
10 ], lr=self.cfg.lr, capturable=True)
11 self.pi_optim = torch.optim.Adam(self.model._pi.parameters(), lr=self.cfg.lr, eps=1e-5,
    capturable=True)

```

Listing 3: tdm2.py: agent 初始化和优化器

9.3 World Model: common/world_model.py

baseline 的 world model 由 encoder、dynamics、reward、policy prior 和 Q ensemble 构成。关键逻辑在 common/world_model.py 第 25-32 行：

```

1 self._encoder = layers.enc(cfg)
2 self._dynamics = layers.mlp(cfg.latent_dim + cfg.action_dim + cfg.task_dim, 2*[cfg.
    mlp_dim], cfg.latent_dim, act=layers.SimNorm(cfg))
3 self._reward = layers.mlp(cfg.latent_dim + cfg.action_dim + cfg.task_dim, 2*[cfg.mlp_dim
    ], max(cfg.num_bins, 1))
4 self._termination = layers.mlp(cfg.latent_dim + cfg.task_dim, 2*[cfg.mlp_dim], 1) if cfg.
    episodic else None
5 self._pi = layers.mlp(cfg.latent_dim + cfg.task_dim, 2*[cfg.mlp_dim], 2*cfg.action_dim)
6 self._qs = layers.Ensemble([layers.mlp(cfg.latent_dim + cfg.action_dim + cfg.task_dim,
    2*[cfg.mlp_dim], max(cfg.num_bins, 1), dropout=cfg.dropout) for _ in range(cfg.num_q)
    ])

```

Listing 4: world_model.py: TD-MPC2 world model 组件

语义对应为：

- encoder: $h(s_t) \rightarrow z_{t0}$
- dynamics: $d(z_t, a_t) \rightarrow z_{t+10}$
- reward: $R(z_t, a_t)$

- pi: policy prior, 用于产生 policy trajectories。
- Qs: Q 函数 ensemble, 用于 planning 末端 bootstrap 和训练 target。

9.4 Planning: tdmpe2.py

act() 会在 `cfg.mpc=true` 时调用 `plan()`, 也就是 `_plan()`:

```
1 if self.cfg.mpc:
2     return self.plan(obs, t0=t0, eval_mode=eval_mode, task=task).cpu()
3 z = self.model.encode(obs, task)
4 action, info = self.model.pi(z, task)
```

Listing 5: tdmpe2.py: 动作选择入口

`_estimate_value()` 用 learned world model 在 latent 空间 rollout, 并在 horizon 末端用 Q bootstrap:

```
1 G, discount = 0, 1
2 for t in range(self.cfg.horizon):
3     reward = math.two_hot_inv(self.model.reward(z, actions[t], task), self.cfg)
4     z = self.model.next(z, actions[t], task)
5     G = G + discount * (1-termination) * reward
6     discount = discount * discount_update
7 action, _ = self.model.pi(z, task)
8 return G + discount * (1-termination) * self.model.Q(z, action, task, return_type='avg')
```

Listing 6: tdmpe2.py: 估计候选动作序列价值

CEM/MPPI 的核心是采样动作序列、选 elite、根据 value soft weighting 更新 mean 和 std:

```
1 r = torch.randn(self.cfg.horizon, self.cfg.num_samples-self.cfg.num_pi_trajs, self.cfg.
    action_dim, device=std.device)
2 actions_sample = mean.unsqueeze(1) + std.unsqueeze(1) * r
3 actions_sample = actions_sample.clamp(-1, 1)
4 actions[:, self.cfg.num_pi_trajs:] = actions_sample
5
6 value = self._estimate_value(z, actions, task).nan_to_num(0)
7 elite_idx = torch.topk(value.squeeze(1), self.cfg.num_elites, dim=0).indices
8 elite_value, elite_actions = value[elite_idx], actions[:, elite_idx]
9
10 max_value = elite_value.max(0).values
11 score = torch.exp(self.cfg.temperature*(elite_value - max_value))
12 score = score / score.sum(0)
13 mean = (score.unsqueeze(0) * elite_actions).sum(dim=1) / (score.sum(0) + 1e-9)
14 std = ((score.unsqueeze(0) * (elite_actions - mean.unsqueeze(1)) ** 2).sum(dim=1) / (
    score.sum(0) + 1e-9)).sqrt()
```

Listing 7: tdmpe2.py: CEM/MPPI elite 更新

最后从 elite 中按 Gumbel-softmax 采样一条动作序列, 执行第一步动作:

```
1 rand_idx = math.gumbel_softmax_sample(score.squeeze(1))
2 actions = torch.index_select(elite_actions, 1, rand_idx).squeeze(1)
3 a, std = actions[0], std[0]
4 if not eval_mode:
5     a = a + std * torch.randn(self.cfg.action_dim, device=std.device)
6 self._prev_mean.copy_(mean)
7 return a.clamp(-1, 1)
```

Listing 8: tdmpe2.py: 选择执行动作

9.5 训练更新: tdmipc2.py

`_update()` 从 replay buffer 取短序列, 训练 dynamics consistency、reward、Q value, 并更新 policy prior。主损失是:

```
1 total_loss = (  
2     self.cfg.consistency_coef * consistency_loss +  
3     self.cfg.reward_coef * reward_loss +  
4     self.cfg.termination_coef * termination_loss +  
5     self.cfg.value_coef * value_loss  
6 )  
7  
8 total_loss.backward()  
9 grad_norm = torch.nn.utils.clip_grad_norm_(self.model.parameters(), self.cfg.  
    grad_clip_norm)  
10 self.optim.step()  
11 self.optim.zero_grad(set_to_none=True)
```

Listing 9: tdmipc2.py: baseline 主损失

policy prior 的更新使用 Q 值作为优化信号:

```
1 action, info = self.model.pi(zs, task)  
2 qs = self.model.Q(zs, action, task, return_type='avg', detach=True)  
3 self.scale.update(qs[0])  
4 qs = self.scale(qs)  
5  
6 rho = torch.pow(self.cfg.rho, torch.arange(len(qs), device=self.device))  
7 pi_loss = -(self.cfg.entropy_coef * info["scaled_entropy"] + qs).mean(dim=(1,2)) * rho).  
    mean()  
8 pi_loss.backward()
```

Listing 10: tdmipc2.py: policy prior 更新

9.6 Replay Buffer: common/buffer.py

Replay buffer 使用 TorchRL 的 ReplayBuffer 和 SliceSampler, 每次采样长度为 horizon+1 的子序列:

```
1 self._sampler = SliceSampler(  
2     num_slices=self.cfg.batch_size,  
3     end_key=None,  
4     traj_key='episode',  
5     truncated_key=None,  
6     strict_length=True,  
7     cache_values=self.cfg.multitask,  
8 )  
9 self._batch_size = self.cfg.batch_size * (self.cfg.horizon+1)  
10  
11 ...  
12  
13 def sample(self):  
14     td = self._buffer.sample().view(-1, self.cfg.horizon+1).permute(1, 0)  
15     return self._prepare_batch(td)
```

Listing 11: buffer.py: 采样 horizon+1 的短轨迹

9.7 Online Training Loop: trainer/online_trainer.py

单任务在线训练逻辑如下: 前 `seed_steps` 用随机动作收集数据, 之后使用 agent planning 选动作, 并开始更新模型。

```

1 if self._step > self.cfg.seed_steps:
2     action = self.agent.act(obs, t0=len(self._tds)==1)
3 else:
4     action = self.env.rand_act()
5 obs, reward, done, info = self.env.step(action)
6 self._tds.append(self.to_td(obs, action, reward, info['terminated']))
7
8 if self._step >= self.cfg.seed_steps:
9     if self._step == self.cfg.seed_steps:
10         num_updates = self.cfg.seed_steps
11         print('Pretraining agent on seed data...')
12     else:
13         num_updates = 1
14     for _ in range(num_updates):
15         _train_metrics = self.agent.update(self.buffer)

```

Listing 12: online_trainer.py: 交互与更新循环

10 复现结论

本次 baseline 复现已经完成以下验证：

1. 成功克隆 TD-MPC2 仓库并建立用户级 tdmpc2 环境。
2. 修复了 torchrl-nightly 不可用、Hydra launcher 缺失、Hugging Face 下载超时等环境问题。
3. DMControl/MuJoCo 在 MUJOCO_GL=egl 下可正常运行。
4. 官方 dog-run-1.pt checkpoint 可正常加载并评估，单 episode reward 为 762.2。
5. 原始 baseline 训练流程已通过 1 万步 smoke test，生成 eval.csv 和 models/final.pt。

需要强调的是，1 万步结果只证明代码链路可用，不代表 TD-MPC2 的最终性能。正式复现 dog-run baseline 应使用 README 示例中的 700 万步或至少更长训练步数，并建议多 seed 报告均值和方差。